

PayLane

A reserve-gated stablecoin treasury on Solana

The program provably cannot mint or settle past on-chain-attested reserves.

Solana / Anchor. Testnet only.

The problem

Fiat-backed stablecoins are the settlement layer of on-chain payments.

But solvency is enforced **off-chain**. The issuer holds reserves at a bank and *promises* that supply never exceeds them.

The mint authority is a key. Whoever holds it can mint past reserves, by error, compromise, or choice.

Holders find out **after** the fact.

Why now

The reserve figure can be attested on-chain today. Minting is already an on-chain action.

The check between them is a **single integer inequality**:

```
circulating + amount <= attested_reserve
```

There is no technical reason the chain cannot refuse an over-mint the way it refuses a double-spend. It just has not been wired into the program that controls the mint.

Scope

A narrow, complete rail, not a broad platform.

- One Anchor treasury program with a reserve gate.
- `mint`, `settle`, `attest_reserve`, separated powers.
- An off-chain agent that runs the same gate before it submits.

We deliberately do **not** try to solve custody or oracles. We move one check to where it belongs.

The solution

A program-derived **treasury account**, bound to one SPL mint, holds `attested_reserve` and a pause flag.

Circulating supply is **not stored**. It is read from `Mint.supply` on every instruction, so the ledger cannot drift from the tokens that exist.

Every `mint` and `settle` keeps the invariant

$\text{Mint.supply} \leq \text{attested_reserve}$

with checked integer math, no floats, and **no admin bypass**.

How it works

```
Attestor --> Treasury acct --> Reserve gate --> SPL mint CPI --> Events
              attested           circ <= res           only if allowed
```

- The gate is a pure-Rust module, `reserve.rs`; the program imports it verbatim.
- `mint` does a real `spl_token::mint_to` CPI signed by the treasury PDA, then re-reads the mint and asserts supply moved by exactly that amount.
- `settle` **burns** the holder's tokens, so supply only falls when tokens disappear.

One source of truth: the code under test is the code that runs on-chain.

Demo

paylane demo

1. Attestor posts proof-of-reserve: **\$1,000,000**.
2. Mint \$300k, then \$500k. Allowed. Circulating \$800k.
3. Try to mint \$300k more, that is \$1.1M against \$1.0M. **REJECTED**. No tx.
4. Settle \$100k, which burns tokens. Headroom frees up.
5. The reserves are audited **down** to \$600k. Recorded, not refused: the treasury **pauses** and every mint is blocked.
6. Holders redeem back under the line. Solvency restored, minting resumes.

The UI shows it: push the slider past the ceiling, or report a reserve loss, and watch the program refuse.

Tech stack

- **Solana / Anchor (Rust)** - the treasury program and the reserve gate. Load-bearing.
- **solana-program-test** - the instructions run against the **real SPL Token program** in an in-process bank. Real supply, real burns, real PDA authority.
- **Pure-Rust invariant module** - unit-tested without the SBF toolchain.
- **agent-core (Python)** - guardrails, audit trail, adapter seam.

Not deployed: **anchor build** needs the Solana toolchain, which is not installed here. No devnet program id, no explorer link. We say so everywhere.

Results

Suite	Result
Rust reserve invariant (cargo test)	39 / 39
Rust program vs. real SPL Token	13 / 13
Python agent + seam (pytest)	44 / 44

The program tests assert on the SPL Mint account, not on anything PayLane wrote about itself. A stubbed mint helper fails all thirteen.

Roadmap

- Deploy to **devnet** and publish an explorer-verifiable mint, burn and pause.
- Pair the gate with a proof-of-reserve **oracle / multi-party attester** to shrink the trusted surface further.
- Threshold attestation instead of a single attester key.
- Multi-currency treasuries with per-currency ceilings.

Solvency enforced by the rail, not promised by the operator.

Thank you

PayLane - reserve-gated stablecoin on Solana.

Dipankar Sarkar / doom2quake

Testnet only. No mainnet, no real funds.